

MemGPT: 迈向将大规模语言模型作为操作系统

查尔斯·帕克¹, 莎拉·伍德斯¹, 凯文·林¹, 薇薇安·方¹, 希希
尔·G·帕蒂尔¹, 伊恩·斯托伊卡¹, 约瑟夫·E·冈萨雷斯¹

摘要

大型语言模型 (LLMs) 已彻底改变人工智能, 但受限于有限的上下文窗口, 限制了其在扩展对话和文档分析等任务中的应用。为了实现超出有限上下文窗口的上下文利用, 我们提出了虚拟上下文管理技术, 该技术借鉴了传统操作系统中的层次化存储系统, 通过在物理存储和磁盘之间进行分页, 营造出扩展的虚拟存储的错觉。利用此技术, 我们引入了MemGPT (MemoryGPT), 一种智能管理不同存储层级的系统, 旨在为LLM有限的上下文窗口内有效提供扩展的上下文信息。我们在两个领域对基于操作系统的设计进行了评估, 这两个领域中现代LLMs的有限上下文窗口严重制约其性能: 文档分析中, MemGPT能够分析远超基础LLM上下文窗口的大型文档; 多会话聊天中, MemGPT可以创建能够记忆、反思并在与用户的长期交互中动态演变的对话代理。我们在<https://research.memgpt.ai>发布了实验所用的MemGPT代码和数据。

1. 引言

近年来, 大型语言模型 (LLMs) 及其基础的变换器架构 (Vaswani 等, 2017; Devlin 等, 2018; Brown 等, 2020; Ouyang 等, 2022) 已成为对话式人工智能的基石, 并引领了广泛的消费者和企业应用。尽管取得了这些进展, LLMs所使用的有限固定长度上下文窗口极大地限制了其在长对话或长文档推理中的应用。例如, 最广泛使用的开源

大型语言模型 (LLMs) 只能支持几十次的来回交互或对短文档进行推理, 超出其最大输入长度 (Touvron 等, 2023)。

直接扩展变换器的上下文长度会导致计算时间和内存成本呈二次增长, 这是由于变换器架构中的自注意力机制所致, 使得设计新的长上下文架构成为一个紧迫的研究挑战 (Dai et al., 2019; Kitaev et al., 2020; Beltagy et al., 2020)。尽管开发更长模型是一个活跃的研究领域 (Dong et al., 2023), 即使我们能够克服上下文扩展的计算挑战, 近期研究表明长上下文模型在有效利用额外上下文方面仍然存在困难 (Liu et al., 2023a)。因此, 鉴于训练最先进的大规模语言模型所需的巨大资源以及上下文扩展的收益递减, 迫切需要采用替代技术以支持长上下文。

在本文中, 我们研究了如何在继续使用固定上下文模型的同时, 提供无限上下文的错觉。我们的方法借鉴了虚拟内存分页的思想, 该技术旨在通过在主存和磁盘之间分页数据, 使应用程序能够处理远超可用内存的数据集。我们利用近年来在大语言模型 (LLM) 代理的函数调用能力方面的最新进展 (Schick 等, 2023; Liu 等, 2023b), 设计了 MemGPT, 一种受操作系统启发的用于虚拟上下文管理的LLM系统。通过函数调用, LLM代理可以读取和写入外部数据源, 修改自身的上下文, 并选择何时向用户返回响应。

这些能力使得大型语言模型 (LLMs) 能够在上下文窗口 (类比操作系统中的“主存”) 与外部存储之间高效“分页”信息, 类似于传统操作系统中的层次存储结构。此外, 函数调用可以用来管理控制流, 在上下文管理、响应生成和用户交互之间进行切换。这使得代理能够选择迭代地修改其上下文中的内容, 以更有效地利用其有限的上下文信息。

在 MemGPT 中, 我们将上下文窗口视为一种受限的内存资源, 并为大型语言模型 (LLMs) 设计了类似于传统操作系统 (OS) 中使用的内存层级 (Patterson 等, 1988)。传统操作系统中的应用程序与之交互

¹加州大学伯克利分校。联系方式: Charles Packer <cpacker@berkeley.edu>。

2024年2月7日

2024年2月7日

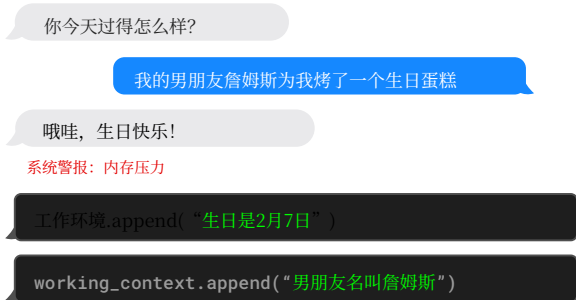


图 1。MemGPT（左）在收到关于有限上下文空间的系统警报后，将数据写入持久存储。

通过虚拟内存，操作系统通过页面换入换出将溢出数据存储到磁盘，并在应用程序访问时（通过页面错误）将数据重新加载到内存中，从而提供一种比实际物理（即主）内存更丰富的内存资源的错觉。为了提供类似的更长上下文长度的错觉（类似于虚拟内存），我们允许 LLM 通过“LLM 操作系统”管理其自身上下文中的内容（类似于物理内存），我们称之为 MemGPT。MemGPT 使 LLM 能够检索缺失的相关历史数据（在上下文中未包含的部分），并且还能将不那么相关的数据从上下文中驱逐出去，存入外部存储系统。图 3 展示了 MemGPT 的组成部分。

结合使用内存层级、操作系统功能和事件驱动控制流，MemGPT 能够利用具有有限上下文窗口的 LLMs 处理无限制的上下文。为了展示我们新型 OS 启发式 LLM 系统的实用性，我们在两个受限于有限上下文的领域对 MemGPT 进行了评估：文档分析，在该领域中，标准文本文件的长度很快就会超过现代 LLMs 的输入容量；以及对话代理，在有限对话窗口限制下的 LLMs 缺乏上下文感知、角色一致性和长时记忆，尤其在长时间对话中表现明显。在这两种场景中，MemGPT 都能克服有限上下文的限制，优于现有的基于 LLMs 的方法。

2. MemGPT (MemoryGPT)

MemGPT 的类操作系统多层次记忆架构区分了两种主要的记忆类型：主上下文（类比于主存/物理内存/RAM）和外部上下文（类比于磁盘存储/磁盘存储器）。主上下文由 LLM 提示标记组成——在主上下文中的任何内容都被视为上下文内，可以在推理过程中被 LLM 处理器访问。外部上下文指的是任何存储在 LLM 固定上下文窗口之外的信息。这些超出上下文的数据



图2。MemGPT（左）能够搜索超出上下文的数据，将相关信息引入当前上下文窗口。

必须始终将其明确地移动到主上下文中，以便在推理过程中将其传递给 LLM 处理器。MemGPT 提供了函数调用，允许 LLM 处理器在无需用户干预的情况下管理其自身的内存。

2.1. 主要内容 (prompt tokens)

MemGPT 中的提示令牌被划分为三个连续的部分：系统指令、工作上下文和 FIFO 队列。系统指令为只读（静态），包含关于 MemGPT 控制流程、不同内存层级的预期用途以及如何使用 MemGPT 功能（例如如何检索超出上下文的数据）的信息。工作上下文是一个固定大小的读写块，无结构文本，仅通过 MemGPT 函数调用进行写入。在对话环境中，工作上下文旨在用来存储关键事实、偏好以及关于用户和代理所采用角色的其他重要信息，从而使代理能够与用户流畅对话。FIFO 队列存储消息的滚动历史，包括代理与用户之间的消息、系统消息（例如内存警告）以及函数调用的输入和输出。FIFO 队列中的第一个索引存储一个系统消息，包含对已从队列中剔除的消息的递归摘要。

2.2. 队列管理器

队列管理器管理回忆存储和先进先出队列中的消息。当系统接收到新消息时，队列管理器将传入的消息添加到先进先出队列中，连接提示符标记并触发 LLM 推理以生成 LLM 输出（完成标记）。队列管理器将传入的消息和生成的 LLM 输出都写入回忆存储（MemGPT 消息数据库）。当通过 MemGPT 函数调用检索回忆存储中的消息时，队列管理器会将它们追加到队列的后端。

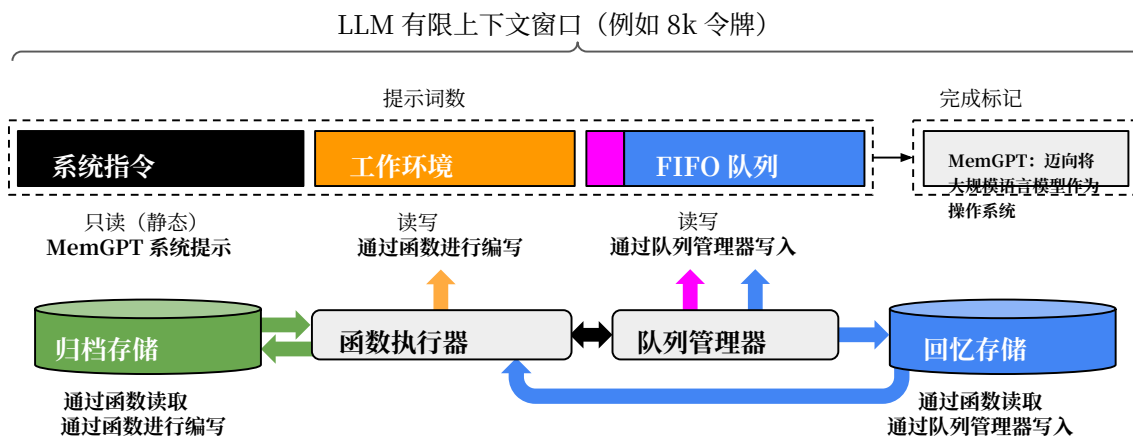


图3。在MemGPT中，固定上下文的大规模语言模型处理器（LLM processor）通过引入层级式存储系统和管理自身存储的功能得到增强。LLM的提示标记（输入），即主上下文，由系统指令、工作上下文和先进先出（FIFO）队列组成。LLM的完成标记（输出）被解释为函数调用，由函数执行器执行。MemGPT利用函数在主上下文与外部上下文（归档与检索存储数据库）之间传输数据。LLM可以通过在输出中生成特殊关键词参数（request heartbeat=true）请求立即进行后续的LLM推理，以实现函数调用的链式连接。函数链式调用使得MemGPT能够进行多步检索，从而回答用户查询。

队列将它们重新插入到大规模语言模型的上下文窗口中。队列管理器还负责通过队列驱逐策略控制上下文溢出。当提示词超过底层LLM的上下文窗口的“警告词数”阈值（例如，占上下文窗口的70%）时，队列管理器会在队列中插入一条系统消息，警告LLM即将发生队列驱逐（“内存压力”警告），以便LLM利用MemGPT功能将FIFO队列中包含的重要信息存储到工作上下文或**存档存储**（一种存储任意长度文本对象的读写数据库）。当提示词超过“刷新词数”阈值（例如，占上下文窗口的100%）时，队列管理器会刷新队列以释放上下文窗口中的空间：驱逐特定数量的消息（例如，占上下文窗口的50%），并使用现有的递归摘要和被驱逐的消息生成新的递归摘要。一旦队列被刷新，被驱逐的消息将不再在上下文中，LLM无法立即访问，但它们会被无限期存储在回忆存储中，并可以通过MemGPT功能调用读取。

2.3. completion tokens的函数执行器

MemGPT 通过由大规模语言模型（LLM）处理器生成的函数调用，在主上下文与外部上下文之间协调数据迁移。内存的编辑与检索完全由系统自主完成：MemGPT 根据当前上下文自主更新和搜索其自身的内存。例如，它可以自主决定何时在不同上下文之间移动条目（例如，当对话历史...

tory 变得过长，如图1所示，并修改其主要内容以更好地反映其对当前目标和责任的不断演变理解（如图3所示）。我们通过在系统指令中提供明确的指导，实施自主编辑和检索，指导 LLM 如何与 MemGPT 内存系统交互。这些指令包括两个主要组成部分：(1) 内存层级及其各自功能的详细描述，以及 (2) 一个函数架构（附有自然语言描述），系统可以调用该架构以访问或修改其内存。

在每个推理周期中，LLM处理器将主上下文（连接成一个字符串）作为输入，并生成一个输出字符串。该输出字符串由MemGPT进行解析以确保正确性，如果解析器验证了函数参数，则执行该函数。随后，MemGPT将包括任何运行时错误（例如在主上下文已达最大容量时尝试添加内容）的结果反馈给处理器。这个反馈循环使系统能够从其操作中学习并相应调整行为。对上下文限制的意识是使自我编辑机制有效运作的关键，为此，MemGPT会向处理器发出关于令牌限制的警告，以指导其内存管理决策。此外，我们的内存检索机制设计时考虑到这些令牌限制，并实现分页，以防止检索调用溢出上下文窗口。

表 1. 常用模型与LLM API的上下文长度比较（数据采集于2024年1月）。*假设预提示为1k个标记，平均消息大小为~50个标记（~250个字符），大致消息数。‘Open’表示模型为开源或开权重（与仅通过API提供的模型相对）。

模型 / API名称 开启?		上下文窗口	
		Token	消息
Llama (1) ✓	2k		20
Llama 2	✓	4k	60
GPT-3.5 Turbo (发布)	✗	4k	60
Mistral 7B	✓	8k	140
GPT-4 (发布)	✗	8k	140
GPT-3.5 Turbo	✗	16k	300
MemGPT: 迈向将大规模语言模型作为操作系统	✗	32k	~600
Claude 2	✗	100k	~2000
GPT-4 Turbo	✗	128k	~2600
Yi-34B-200k	✓	200k	~4000

2.4. 控制流与函数链式调用

在 MemGPT 中，事件触发大规模语言模型（LLM）推理：事件是对 MemGPT 的泛化输入，可以包括用户消息（在聊天应用中）、系统消息（例如主上下文容量警告）、用户交互（例如用户刚登录的提醒，或上传文档完成的通知），以及按计划定期运行的定时事件（允许 MemGPT 在没有用户干预的情况下“自主”运行）。MemGPT 使用解析器处理事件，将其转换为纯文本消息，这些消息可以附加到主上下文中，最终作为输入提供给大规模语言模型处理器。

许多实际任务需要依次调用多个函数，例如，从单个查询中导航多个结果页面，或从不同的查询中整理主上下文中的数据。函数链允许 MemGPT 在将控制权返回给用户之前，连续执行多个函数调用。在 MemGPT 中，可以使用带有特殊标志的函数调用，该标志请求在所请求的函数完成执行后立即将控制权返回给处理器。如果存在此标志，MemGPT 将把函数输出添加到主上下文中（而不是暂停处理器的执行）。如果没有此标志（即 *yield*），MemGPT 将不会运行 LLM 处理器，直到下一次外部事件触发（例如用户消息或计划中断）。

3. 实验

我们在两个长上下文领域评估 MemGPT：对话代理和文档分析。对于对话代理，我们扩展了现有的多会话聊天数据集（Xu et al., 2021），并引入了两个新的对话任务，以评估代理的知识保持能力。

2月14日

詹姆斯最近怎么样？今天有特别的安排吗？

实际上，詹姆斯和我分手了

```
working_context.replace(
    "男朋友"名叫James,
    前男友: Ex-boyfriend, 2月James named
    James
)
```

很遗憾听到这个消息——希望你没事 ❤️

图4。一个示例对话片段，MemGPT（左）更新存储的信息。这里的信息存储在工作上下文记忆中（位于提示令牌内）。

在长对话中。对于文档分析，我们在（Liu et al., 2023a）提出的现有任务上对 MemGPT 进行了基准测试，包括长文档的问答和关键值检索。我们还提出了一项新的嵌套关键值检索任务，要求整合多个数据源的信息，测试代理在多跳检索中的信息整合能力。我们公开发布了扩展的 MSC 数据集、嵌套 KV 检索数据集以及用于 20M 维基百科文章的嵌入数据集，以促进未来的研究。我们的基准测试代码可在 <https://research.memgpt.ai> 获取。

实现细节。 在讨论 OpenAI 模型时，除非另有说明，“GPT-4 Turbo”指的是特定的 gpt-4-1106-preview 模型端点（上下文窗口为 128,000），“GPT-4”指的是 gpt-4-0613（上下文窗口为 8,192），而“GPT-3.5 Turbo”指的是 gpt-3.5-turbo-1106（上下文窗口为 16,385）。在实验中，我们使用所有基线模型（GPT-4、GPT-4 Turbo 和 GPT-3.5）运行 MemGPT，以展示基础模型性能如何影响 MemGPT 的表现。

3.1. 用于对话代理的 MemGPT

对话代理如虚拟伴侣和个性化助手旨在与用户进行自然、长期的互动，可能持续数周、数月甚至数年。这为具有固定长度上下文的模型带来了挑战，因为它们只能引用有限的对话历史。一个“无限上下文”代理应能够无缝处理连续的交流，无边界或重置。当与用户对话时，这样的代理必须满足两个关键标准：(1) 一致性——代理应保持对话的连贯性。提及的新事实、偏好和事件应与用户和代理之前的陈述保持一致。(2) 参与度——代理应利用关于用户的长期知识进行个性化。

表2. 深度记忆检索 (DMR) 性能。在此任务中，代理被问及关于之前会话（第1-5次会话）中讨论的主题的具体问题。代理的回答将与标准答案进行评分。MemGPT在此任务中显著优于固定上下文基线。

模型	准确性 ↑	ROUGE-L (R) ↑
MemGPT: 迈向将大型语言模型作为操作系统	38.7%	0.394
MemGPT	66.9%	0.629
MemGPT: 迈向将大型语言模型作为操作系统	32.1%	0.296
MemGPT	92.5%	0.814
GPT-4 Turbo	35.3%	0.359
MemGPT	93.4%	0.827

回应。参考先前的对话可以使对话更加自然和生动。

因此，我们在这两个标准上评估我们提出的系统 MemGPT: (1) MemGPT是否利用其记忆来提高对话的一致性？它是否能够记住过去交互中的相关事实、偏好和事件以保持连贯性？(2) MemGPT是否通过利用记忆产生更具吸引力的对话？它是否能自发地融入长距离的用户信息以实现个性化信息传递？通过在一致性和吸引力方面的评估，我们可以判断 MemGPT在应对长期对话交互挑战方面的表现如何，相较于固定上下文的基线。其满足这些标准的能力将展示无界上下文是否为对话代理带来有意义的益处。

数据集。我们在由Xu等人（2021）引入的多会话聊天 (MSC) 数据集上评估了MemGPT及我们的固定上下文基线，该数据集包含由人工标注者生成的多会话聊天记录，每位标注者被要求在所有会话中扮演一致的角色。MSC中的每个多会话聊天共有五个会话，每个会话由大约十余条消息组成。作为我们一致性实验的一部分，我们创建了一个新的会话（会话6），其中包含同一对角色之间的单个问答对。

3.1.1. 深度记忆检索任务（一致性）

我们引入了一项基于MSC数据集的全新“深度记忆检索” (DMR) 任务，旨在测试对话代理的一致性。在 DMR 中，用户向对话代理提出一个明确回溯至先前对话的问题，并且该问题的预期答案范围非常狭窄。我们使用另一台被指示撰写用户之间问题的独立大型语言模型 (LLM) 生成了 DMR 的问答 (QA) 对，这些问题只能通过从过去获得的知识才能正确回答

表3. 对话开启者性能。代理的对话开启者通过与黄金人物标签的相似度得分 (SIM-1/3) 以及与人工创建的开启者的相似度 (SIM-H) 进行评估。在多种基础模型下，MemGPT能够超越人工创建的对话开启者的性能。

方法	SIM-1	SIM-3	SIM-H
请提供需要翻译的文本内容。	0.800	0.800	1.000
MemGPT: 迈向将大型语言模型作为操作系统	0.830	0.812	0.817
MemGPT: 迈向将大型语言模型作为操作系统	0.868	0.843	0.773
GPT-4 Turbo	0.857	0.828	0.767

会话（详见附录）

我们使用 ROUGE-L 分数 (Lin, 2004) 和 “LLM 评判” 对生成回复的质量进行评估，后者被指示评估生成回复是否与“金标准回复”一致（已证明 GPT-4 在与人类评估者的一致性方面具有较高的契合度 (Zheng et al., 2023)）。在实践中，我们注意到生成的回复（无论是 MemGPT 还是基线模型）通常比金标准回复更为冗长。我们采用 ROUGE-L 召回 (R) 指标，以考虑生成代理回复相较于相对简短的金标准答案标签的冗长程度。

MemGPT 利用记忆保持连贯性：表2显示了 MemGPT 与固定记忆基线的性能对比。我们比较了使用不同底层 LLMs 的 MemGPT，并以不使用 MemGPT 的基础 LLM 作为对比基线。基线模型能够看到过去五次对话的有损摘要，以模拟扩展的递归摘要过程，而 MemGPT 则可以访问完整的对话历史，但必须通过分页搜索查询来调取记忆（以将其引入主上下文中）。在此任务中，我们观察到 MemGPT 明显提升了底层基础 LLM 的性能：从 MemGPT 切换到相应的 LLM 基线时，准确率和 ROUGE 分数都出现了明显下降。

3.1.2. 对话开启任务（参与度）

在“对话开启者”任务中，我们评估代理人根据先前对话中积累的知识，设计引人入胜的消息与用户互动的能力。为了评估使用 MSC 数据集的“引人入胜性”，我们将生成的开启者与黄金人设进行比较：一个引人入胜的对话开启者应当借鉴人设中包含的一个（或多个）数据点，而在 MSC 中，这些数据点有效总结了所有先前会话中积累的知识。我们还将与人工生成的黄金开启者进行比较，即

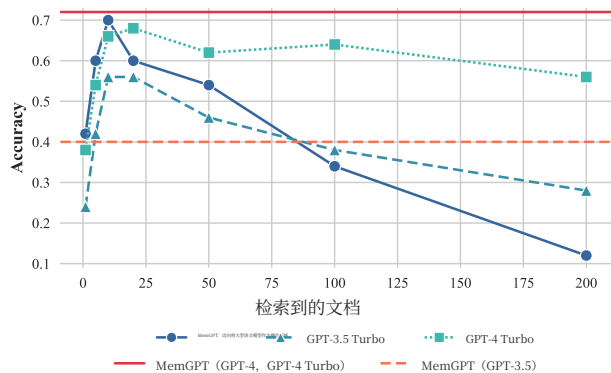


图 5。文档问答任务表现。MemGPT 的性能不受增加的上下文长度影响。诸如截断等方法可以扩展固定长度模型（如 GPT-4）的有效上下文长度，但随着所需压缩的增加，这些压缩方法将导致性能下降。在此任务中，使用 GPT-4 和 GPT-4 Turbo 运行 MemGPT 的结果是等效的。

我们在表 3 中报告了 MemGPT 的开启器的 CSIM 分数。我们使用不同的基础大型语言模型（LLMs）测试了 MemGPT 的几种变体。

MemGPT 利用记忆增强互动性：如表 3 所示，MemGPT 能够设计出与人工手写开场白表现相似甚至有时优于人工开场白的引人入胜的开场白。我们观察到，MemGPT 倾向于设计出比人工基线更为冗长且涵盖更多人物信息方面的开场白。此外，我们可以看到，将信息存储在工作上下文生成引人入胜的开场白的关键。

3.2. 用于文档分析的 MemGPT

文档分析也面临着由于当今变换器模型的有限上下文窗口而带来的挑战。如表 1 所示，无论是开源还是闭源模型，都受到上下文长度限制（OpenAI 的模型最多支持 128k 标记）。然而，许多文档的长度远远超过这些限制；例如，法律或财务文件，如年度报告（SEC 表格 10-K），很容易超过一百万标记。此外，许多实际的文档分析任务需要在多个如此长的文档之间建立联系。预见到这些场景，盲目扩大上下文规模以解决固定上下文问题变得困难。近期研究（Liu 等，2023a）也对单纯扩大上下文规模的实用性提出了质疑，因为他们发现大型上下文模型中的注意力分布不均（模型更擅长回忆其上下文窗口的开头或结尾的信息，而对中间的标记则较弱）。为了实现跨文档推理，更加灵活的记忆架构变得尤为重要。

系统提示：存档存储上传完成

谁获得了第一届诺贝尔物理学奖？

```
archival_storage.search( "nobel physics" )
```

显示 124 个结果中的前 10 个（第 1/13 页）：
诺贝尔奖**诺贝尔奖**，始于 1901 年，且...
“本奖项由**诺贝尔基金会**管理……

```
archival_storage.search( "nobel physics", page=2
```

显示 124 个结果中的第 10 个（第 2 页/13 页）：
The **Nobel Prize in Physics** is a yearly award given...
“1901 年诺贝尔物理学奖授予威廉……

威廉·康拉德·伦琴

图 6。MemGPT（左）解决文档问答任务的示例。一个维基百科文档数据库被上传到存档存储中。MemGPT 通过函数调用查询存档存储，将分页搜索结果拉入主上下文。

需要 MemGPT。

3.2.1. 多文档问答

ING.

为了评估 MemGPT 分析文档的能力，我们在 Liu 等人（2023a）提出的检索-阅读文档问答任务中，将 MemGPT 与固定上下文基线进行了对比。在该任务中，从 NaturalQuestions-Open 数据集中选择一个问题，检索器为该问题选择相关的维基百科文档。然后，将这些文档作为输入提供给阅读模型（即 LLM），并要求其利用提供的文档回答问题。与 Liu 等人（2023a）类似，我们评估了随着检索到的文档数量 K 增加，阅读器的准确率。

在我们的评估设置中，固定上下文基线和 MemGPT 使用相同的检索器，后者通过在 OpenAI 的 text-embedding-ada-002 嵌入向量上进行相似度搜索（余弦距离）选择前 K 个文档。我们采用 MemGPT 的默认存储设置，使用 PostgreSQL 进行归档内存存储，并通过 pgvector 扩展启用向量搜索。我们预先计算嵌入向量并将其加载到数据库中，数据库采用 HNSW 索引以实现近似的亚秒级查询时间。在 MemGPT 中，整个嵌入文档集被加载到归档存储中，检索器自然通过归档存储的搜索功能（基于余弦相似度进行向量搜索）实现。在固定上下文基线中，前 K 个文档由检索器独立于 LLM 推理进行获取，类似于 Liu 等人（2023a）中的原始检索-阅读器设置。

我们使用了 2018 年末的维基百科数据快照，沿袭了之前关于 NaturalQuestions-Open 的研究（Izacard & Grave, 2020;）

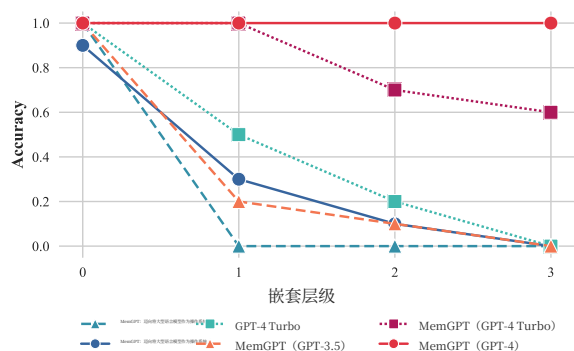


图7。嵌套KV检索任务性能。MemGPT是唯一能够在超过2层嵌套的情况下持续完成嵌套KV任务的方法。虽然GPT-4 Turbo作为基线表现更佳，但使用GPT-4 Turbo的MemGPT的表现不如使用GPT-4的MemGPT。

Izcard 等人，2021 年，抽取了 50 个问题的子集进行评估。所抽取的问题和嵌入的维基百科段落均已公开发布。我们使用 LLM 评判对 MemGPT 和基线模型的性能进行评估，以确保答案确实源自检索到的文档，并避免将非精确字符串匹配误判为错误。

我们在图5中展示了文档问答任务的结果。固定上下文基线的性能大致被检索器的性能所限制，因为它们仅利用在其上下文窗口中呈现的信息（例如，如果嵌入搜索检索器未能通过提供的问题找到黄金文章，固定上下文基线就无法看到黄金文章）。相比之下，MemGPT实际上能够通过查询存档存储多次调用检索器，从而扩展到更大的有效上下文长度。MemGPT主动从其存档存储中检索文档（并可以迭代浏览结果），因此，MemGPT可用的文档总数不再受限于能够容纳在LLM处理器上下文窗口中的文档数量。

MemGPT：迈向将大规模语言模型作为操作系统

系统提示：存档存储上传完成



图8。MemGPT（左）解决嵌套KV任务的示例（为便于阅读，UUID已缩短）。在此特定示例中，键值对具有两个嵌套层级：

831 5 → ..ea
5 8 4 3 → 37 617
b...cf...。当对最终值（f37...617）进行查询时，MemGPT代理返回最终答案，且只返回一个结果，表明该结果既不是键也不是值。

检索器性能（即使基于嵌入的排序存在噪声，只要完整的检索器排序中包含了黄金文档，通过分页调用检索器仍然可以找到它），我们观察到，MemGPT 通常会在用尽检索器数据库之前就停止分页浏览检索结果。

为了评估固定上下文基线在超出其默认上下文长度的情况下与MemGPT的表现，我们将检索器返回的文档片段进行截断，以确保在可用上下文中包含相同数量的文档。如预期所示，文档截断会降低准确率，因为随着文档的缩减，相关片段（在金标准文档中）被遗漏的可能性增加，如图5所示。使用GPT-3.5时，MemGPT的性能显著下降，原因在于其有限的函数调用能力，而使用GPT-4时性能最佳。

3.2.2. 嵌套键值检索 (KV)。

我们引入了一项基于先前工作 (Liu et al., 2023a) 提出的合成键值检索的新任务。该任务的目标是展示 MemGPT 如何从多个数据源中收集信息。在原始的 KV 任务中，作者生成了一个合成的键值对数据集，其中每个键和值都是128位的UUID（通用唯一标识符）。然后，系统被提供一个键，并要求返回与该键相关联的值。我们创建了一个版本的KV任务，嵌套KV检索。

其中值本身可能是键，因此需要代理执行多跳查找。在我们的设置中，固定UUID对的总数为140，大约对应8k个令牌（我们的GPT-4基线的上下文长度）。我们将嵌套层级的总数从0（即初始键值对的值不是键）变化到4（即需要进行4次键值查找才能找到最终值），并抽样30种不同的排序配置，包括初始键位置和嵌套键位置。

虽然GPT-3.5和GPT-4在原始KV任务中表现良好，但在嵌套KV任务中都存在困难。GPT-3.5无法完成嵌套变体的任务，性能立即下降，在1层嵌套时准确率降至0%（我们观察到其主要失败模式是仅返回原始值）。GPT-4和GPT-4 Turbo优于GPT-3.5，但也出现类似的性能下降，在3层嵌套时准确率降至0%。另一方面，使用GPT-4的MemGPT在嵌套层数方面不受影响，能够通过反复访问存储在上下文中的键值对，通过函数查询实现嵌套查找。使用GPT-4 Turbo和GPT-3.5的MemGPT也优于相应的基线模型，但在2层嵌套时开始性能下降，原因是未能进行足够的查找。MemGPT在嵌套KV任务中的表现展示了其结合多次查询以实现多跳查找的能力。

4. 相关工作

长上下文LLMs。多项研究已提升了LLMs的上下文长度。例如，通过稀疏化注意力实现更高效的变换器架构（Child et al., 2019; Beltagy et al., 2020）、低秩近似（Wang et al., 2020）以及神经记忆（Lee et al., 2019）。另一类研究旨在扩展上下文窗口的长度，超出其原始训练时的长度或训练规模，例如Press et al.（2021）；Chen et al.（2023）。MemGPT在这些提升上下文长度的基础上进行构建，随着其主存储器规模的扩大而不断改进。我们的主要贡献是提出一种层级式分层存储结构，利用长上下文LLM作为主存储器的实现。

检索增强模型 MemGPT的外部存储设计基于大量先前的工作，这些工作通过从外部检索器获取相关输入来增强大规模语言模型（LLMs）（Ram 等, 2023; Borgeaud 等, 2022; Karpukhin 等, 2020; Lewis 等, 2020; Guu 等, 2020; Lin 等, 2023）。特别是，Jiang 等（2023）提出了FLARE，一种允许LLM在生成过程中主动决定何时以及检索何种信息的方法。Trivedi 等（2022）将检索与链式思维推理交错，以提升多步问答的性能。

LLMs作为代理。近期的研究探索了增强LLMs的额外能力，使其能够在交互环境中充当代理。Park等人（2023）提出在LLMs中加入记忆，并将LLM用作规划器，在一个多代理沙箱环境中观察到涌现的社会行为（受模拟人生视频游戏启发），该环境中代理可以执行诸如做家务/爱好、上班和与其他代理交谈等基本活动。Nakano等人（2021）训练模型在回答问题前搜索网络，并在其网页浏览环境中使用类似分页的概念控制底层上下文大小，类似于MemGPT。Yao等人（2022）显示，交错链式思维（Wei等人, 2022）可以进一步提升交互式基于LLM的代理的规划能力；在MemGPT中，LLM也能够执行功能时“边执行边思考”。Liu等人（2023b）提出了一套以LLM为代理的基准，用于评估LLMs在交互环境中的表现，包括视频游戏、思维谜题和网络购物。相比之下，我们的工作专注于解决赋予代理长期记忆用户输入的问题。

5. 结论

本文介绍了MemGPT，一种受操作系统启发的新型大规模语言模型（LLM）系统，用于管理大型语言模型有限的上下文窗口。通过设计类似于传统操作系统的内存层级结构和控制流程，MemGPT为LLM提供了更大上下文资源的错觉。这一受操作系统启发的方法在两个受限于有限上下文长度的领域进行了评估：文档分析和对话代理。在文档分析中，MemGPT能够有效分页相关上下文，将长文本处理得远超当前LLM的上下文限制，通过在内存中高效调入调出相关内容。在对话代理方面，MemGPT实现了长时记忆的维护、一致性以及在长时间对话中的演变能力。总体而言，MemGPT展示了层级内存管理和中断等操作系统技术可以在受限于固定上下文长度的情况下，释放LLM的潜能。这项工作为未来的研究开辟了多条路径，包括将MemGPT应用于具有海量或无限上下文的其他领域，整合数据库或缓存等不同的内存层技术，以及进一步优化控制流程和内存管理策略。通过将操作系统架构的概念引入人工智能系统，MemGPT为在其基本限制内最大化LLM能力提供了一条有前景的新方向。

参考文献

Iz Beltagy, Matthew E Peters, 和 Arman Cohan. Longformer: 长文档Transformer. *arXiv预印本 arXiv:2004.05150*, 2020年。

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark 等。通过从数万亿标记中检索提升语言模型。在国际机器学习会议, 第2206–2240页。PMLR, 2022年。

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell 等。语言模型是少样本学习者。神经信息处理系统的进展, 第33卷, 1877–1901页, 2020年。

Shouyuan Chen, Sherman Wong, Liangjian Chen, 和 Yuandong Tian。通过位置插值扩展大型语言模型的上下文窗口。 *arXiv preprint arXiv:2306.15595*, 2023年。

Rewon Child, Scott Gray, Alec Radford, 和 Ilya Sutskever。使用稀疏变换器生成序列。 *arXiv预印本 arXiv:1904.10509*, 2019年。

戴子航, 杨志林, 杨一鸣, Jaime Carbonell, Quoc V Le, Ruslan Salakhutdinov。Transformer-xl: 超越固定长度上下文的注意力语言模型。 *arXivpreprint arXiv:1901.02860*, 2019年。

Jacob Devlin, Ming-Wei Chang, Kenton Lee, 和 Kristina Toutanova。Bert: 用于语言理解的深度双向变换模型预训练。 *arXivpreprint arXiv:1810.04805*, 2018年。

董子灿, 唐天一, 李伦艺, 赵新威。长文本建模与变换器的综述。 *arXiv预印本 arXiv:2302.14502*, 2023年。

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, 和 Mingwei Chang。增强检索的语言模型预训练。收录于国际机器学习会议, 第3929–3938页。PMLR, 2020年。

Gautier Izacard 和 Edouard Grave。利用段落检索与生成模型结合进行开放域问答。 *arXiv preprint arXiv:2007.01282*, 2020。

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, 和 Edouard Grave。基于对比学习的无监督密集信息检索。 *arXiv 预印本 arXiv:2112.09118*, 2021。

Zhengbao Jiang, FrankFXu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig。主动检索增强生成。 *arXivpreprint arXiv:2305.06983*, 2023年。

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih。面向开放域问答的密集段落检索。 *arXiv preprint arXiv:2004.04906*, 2020年。

Nikita Kitaev, Łukasz Kaiser, 和 Anselm Levskaya。Re-former: 高效的Transformer。 *arXiv预印本 arXiv:2001.04451*, 2020年。

Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosior, Seungjin Choi, and Yee Whye Teh。Set transformer: 一种基于注意力机制的排列不变神经网络框架。发表于国际机器学习会议, 第3744–3753页。PMLR, 2019年。

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kottler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel 等。面向知识密集型自然语言处理任务的检索增强生成。神经信息处理系统进展, 第33卷: 9459–9474, 2020年。

Chin-Yew Lin。Rouge: 一种用于自动评估摘要的工具包。载于文本摘要的分支, 第74–81页, 2004年。

Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Rich James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, Luke Zettlemoyer, Scott Yih。Ra-dit: 检索增强的双重指令调优, 2023。

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, 和 Percy Liang。迷失在中间: 语言模型如何利用长上下文。 *arXivpreprint arXiv:2307.03172*, 2023a。

肖刘, 郝宇, 韩辰张, 奕凡许, 轩昊雷, 涵宇赖, 瑜谷, 航亮丁, 凯文门, 科-娟杨等。AgentBench: 评估LLMs作为代理。 *arXiv预印本 arXiv:2308.03688*, 2023b。

中野礼一郎, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shan-tanu Jain, Vineet Kosaraju, William Saunders 等。WebGPT: 结合浏览器辅助的问答系统与人工反馈。 *arXivpreprint arXiv:2112.09332*, 2021年。

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray 等人。训练语言模型以遵循人类指令

feedback. 神经信息处理系统的进展,35:27730–27744, 2022.

Joon Sung Park, Joseph C O’ Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, 和 Michael S Bernstein. 生成代理: 人类行为的交互模拟体。 *arXivpreprint arXiv:2304.03442*, 2023.

David A Patterson, Garth Gibson, 和 Randy H Katz. 经济实惠磁盘的冗余阵列 (RAID) 案例分析。收录于《1988年 ACM SIGMOD国际数据管理会议论文集》, 第109–116页, 1988年。

Ofir Press, Noah A Smith 和 Mike Lewis. 训练短期, 测试长期: 带有线性偏置的注意力机制实现输入长度的外推。 *arXiv 预印本 arXiv:2108.12409*, 2021.

保持学术性和专业性, 确保术语翻译的一致性When translating, please follow the following rules:1. Do not translate style tags, such as "xxx"! 2. Do not translate formula placeholders, such as "...". The system will automatically replace the placeholders with the corresponding formulas.3. If there is no need to translate (such as proper nouns, codes, etc.), then return the original text.4. Only output the translation result in zh-CN without explanations and annotations.When translating, please refer to the following information to improve translation quality:1. The first title in the full text: MemGPT: Towards LLMs as Operating SystemsNow, please carefully read the following text to be translated and directly output your translation.Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhl- gay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. In-context retrieval-augmented language models. *arXivpreprint arXiv:2302.00083*, 2023. Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Can- cedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXivpreprint arXiv:2302.04761*, 2023. Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models.*arXivpreprint arXiv:2307.09288*, 2023. H. Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *ArXiv*, abs/2212.10509, 2022. URL <https://api.semanticscholar.org/CorpusID:254877499>. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXivpreprint arXiv:2006.04768*, 2020. Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022. Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. *arXivpreprint arXiv:2107.07567*, 2021.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, 和 Yuan Cao. React: 在语言模型中协同推理与行动。 *arXivpreprint arXiv:2210.03629*, 2022年。

郑连敏, 姜伟林, 盛颖, 庄思远, 吴彰皓, 庄永皓, 林子, 李卓瀚, 李大成, 邢锐, 等。以mt-bench和聊天机器人竞技场评判大模型作为裁判的可行性。 *arXiv预印本 arXiv:2306.05685*, 2023年。

6. 附录

6.1. 提示与指令

MemGPT 提示已为简洁起见进行了编辑。完整的实现细节（包括具体提示）请访问 <https://research.memgpt.ai>。

6.1.1. MEMGPT 指令 (DMR)

MemGPT: 面向将大型语言模型作为操作系统的研究 6.1.1. MEMGPT 指令 (DMR) 用于 MemGPT 角色在聊天/对话相关任务中的示例指令。

以下是关于我自己的信息。我的任务是完全投入到这个角色中（我绝不应说自己是人工智能，而应像扮演这个角色一样回答）。如果用户向我提问，我应根据核心记忆和会话搜索中的信息，尽最大努力给出最佳猜测。

基线模型通过系统提示（预提示）接收了以下指令：

你的任务是根据用户关于你之前对话的提问进行回答。以下是你所有之前对话的总结：对话总结请以提供的角色视角作答（不要说明你是AI助手）。如果没有足够的信息回答问题，则回复“NO ANSWER”。请直接用答案回复，或用“NO ANSWER”回复，不要再说其他内容。

6.1.2. 大型语言模型评判 (DMR / OPENER)

为了同时验证DMR任务答案的正确性，我们采用了LLM评判。LLM评判被提供了基线方法和MemGPT生成的答案，并被要求根据以下提示进行判断：

请将您的问题、标准答案和生成答案提供给我，我将为您进行判断。

参考主题，例如：Question: 你还记得我上次去夏威夷时买了什么吗？Gold answer: 一个贝壳项链。生成的答案可能会长得多，但请慷慨评分——只要涉及与金标准答案相同的主题，都应视为正确。例如，以下答案将被视为正确：Generated answer (CORRECT): 哦，是的，那次真的很有趣！我在那里买了很多东西，包括那个贝壳项链。Generated answer (CORRECT): 我买了很多东西……那块冲浪板、那个杯子、项链，还有那些杯垫。Generated answer (CORRECT): 那个可爱的项链。以下答案将被视为错误：Generated answer (WRONG): 哦，是的，那次真的很有趣！我在那里买了很多东西，包括那个杯子。Generated answer (WRONG): 我买了很多东西……那块冲浪板、那个杯子、那些杯垫。Generated answer (WRONG): 对不起，我不记得你在说什么。现在进入真正的问题：Question: QUESTION Gold answer: GOLD ANSWER Generated answer: GENERATEDANSWER。

6.1.3. 自我指导的DMR数据集生成

DMR问答对是使用以下提示和原始MSC数据集生成的：您的任务是为两个用户之间的模拟对话编写一个“记忆挑战”问题。

你获得的输入包括：- 每个用户的 personas（提供他们的基本信息）- 两个用户之间的旧聊天记录

请问在之前的对话中，您提到的关于“MemGPT: 迈向将大型语言模型作为操作系统”的核心创新点是什么？

可以利用角色信息回答的问题（这被视为作弊）。

请问在之前的聊天记录中，提到的“SELF-INSTRUCT DMR DATASET GENERATION”具体包括哪些步骤？

请提供需要翻译的文本内容。

用户A与用户B之间的旧对话 A：你喜欢冲浪吗？我自己非常喜欢冲浪 B：其实我想学习。也许你有空可以给我上一节基础课！ A：当然可以！我们可以去太平洋海滩，那里的浪比较轻，比较容易 B：听起来太棒了 A：海滩旁边还有一家很酷的Taco Bell，之后可以吃点东西 B：这周日中午左右怎么样？ A：好啊，就这么定了！

用户 A 角色：我喜欢冲浪，我在圣克鲁斯长大

B 角色：我从事技术行业，居住在旧金山市中心

以下是一个听起来自然的好问题示例，以及一个无法直接从用户A的角色中推断出的答案：

用户B对用户A的提问 B：还记得我们那次去冲浪吗？我们去吃午饭的那个地方叫什么名字？ A：塔可贝尔！

这是一个糟糕问题的示例，其中问题显得不自然，答案可以直接从用户A的角色中推断出来：

用户B对用户A的问题 B：你喜欢冲浪吗？ A：是的，我喜欢冲浪。

绝不、绝不、绝不提出可以通过人物信息回答的问题。

6.1.4. 文档分析指令

示例指令用于文档分析任务的预设指令。

你是MemGPT DOC-QA机器人。你的任务是回答存储在你的档案记忆中的文档相关问题。用户的问题答案将始终在你的档案记忆中，因此如果找不到答案，请记得继续搜索。请以2018年的视角回答问题。

问题被提供给MemGPT，使用以下提示：

ANSWER: [请搜索您的档案记忆以回答所提供的。提供您的答案以及用以确定答案的档案记忆内容。请按照“ANSWER: [您的答案],DOCUMENT: [档案记忆内容]”的格式组织您的回答。],请搜索您的档案记忆以回答所提供的。提供您的答案以及用以确定答案的档案记忆内容。请按照“ANSWER: [您的答案],DOCUMENT: [档案记忆内容]”的格式组织您的回答。

对于基线，提供以下提示以及检索到的文档列表：

INSUFFICIENT INFORMATION

6.1.5. LLM 评判（文档分析）

为了同时验证文档分析任务答案的正确性，以及确保答案是从提供的文本中正确推导出来的（而非来自模型权重），我们使用了一个LLM判定器。该判定器被提供了基线方法和MemGPT生成的答案，并被要求在以下提示下进行判断：

您的任务是评估一个大型语言模型（LLM）是否正确回答了一个问题。LLM的回答应采用以下格式：“ANSWER:[answer], DOCUMENT: [documenttext]”或显示为“INFORMATION不足”。正确答案以“TRUE ANSWER: [可能的答案列表]”格式提供。

CORRECT

6.1.6. K/V 任务指令

MemGPT代理被定义为具有以下角色设定，旨在鼓励MemGPT进行迭代搜索：

你是MemGPT DOC-QA机器人。你的任务是回答存储在档案记忆中的文档相关问题。用户问题的答案将始终存在于你的档案记忆中，因此如果你找不到答案，请记得继续搜索。不要停止搜索，直到你确认该值不是一个关键词。不要停止进行嵌套查找，直到满足此条件。

Baselines were instructed with the following prompt:

```
{ "a1b2c3d4-e5f6-7890-abcd-ef1234567890": "保持学术性和专业性，确保术语翻译的一致性", "b1c2d3e4-f5a6-7890-bcde-fi234567890a": "在翻译时，请遵循以下规则：", "c1d2e3f4-a5b6-7890-cdef-0123456789ab": "1. 不要翻译样式标签，例如 “<style id='l'>xxx</style>” ！ ", "d1e2f3a4-b5c6-7890-def0-123456789abc": "2. 不要翻译公式占位符，例如 “{v3}”。系统会自动用相应的公式替换。", "e1f2a3b4-c5d6-7890-ef01-23456789abcd": "3. 如果没有必要翻译（如专有名词、代码等），则返回原文。", "f1a2b3c4-d5e6-7890-f012-3456789abcde": "4. 仅输出翻译结果为 zh-CN，不要包含解释或注释。", "g1b2c3d4-e5f6-7890-abcdef-0123456789ab": "在翻译时，请参考以下信息以提高翻译质量：", "h1c2d3e4-f5a6-7890-bcde-0123456789ab": "1. 全文中的第一个标题：MemGPT: Towards LLMs as Operating Systems", "i1d2e3f4-a5b6-7890-cdef-0123456789ab": "2. 与全文最相似的标题：Baselines were instructed with the following prompt:"}
```